SRM INSTITUTE OF SCIENCE & TECHNOLOGY

NCR CAMPUS, MODINAGAR

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATION

PRACTICAL FILE

Operating System [PCA20C02J]

MCA 1st YEAR, 1st SEMESTER

Session: 2023-24

Submitted To:

Mrs. Rachna Sharma
(ASSISTANT PROFESSOR)
MCA DEPT.

Submitted By:

SRM INSTITUTE OF SCIENCE&TECHNOLOGY

NCR CAMPUS, MODINAGAR

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATION

Register No.:

BONAFIDE CERTIFICATE

Certified to be the Bonafide record of the work done by_____ of MCA 1st year, 1st Semester for the award of **Master's** degree course in the FACULTY OF SCIENCE & HUMANITIES in DEPARTMENT OF COMPUTER APPLICATION in **Operating System [PCA20C02J]** laboratory during the Academic year-2023-24.

LAB IN-CHARGE **HEAD- COMPUTER APPLICATION**

Submitted for the university examination held on_____

INTERNAL EXAMINER 1 **INTERNAL EXAMINER 2**

S.NO.	TITLE OF EXPERIMENT	Date of Execution	Page No	Signature / Remarks
1	Understanding the booting process of Linux.	**27/07/2023**		
2	Understanding the behaviour of the OS(Linux)and get the CPU type model.	**03/08/2023**		
3	Understanding the Linux file system commands.	**03/08/2023**		
4	Write a program to read a file and display its content.	**14/08/2023**		
5	Understanding various phases of Compilation and System admin commands.	**16/08/2023**		
6	Understanding the Linux commands.	**16/08/2023**		
7	Write a program to create child process using fork () system call.	**18/08/2023**		
8	Write a program to implement the concept of thread.	**22/08/2023**		
9	Write a program to implement write operation of message passing in inter-process communication.	**12/09/2023**		
10	Write a program to implement read operation of message passing in inter-process communication.	**14/09/2023**		
11	Write a program to implement pipe() function.	**16/09/2023**		
12	To write a program to implement the concept of Semaphore.	**18/09/2023**		
13	Write a program to implement the Producer Consumer Problem.	**25/09/2023**		
14	Write a shell program to implement While loop.	**25/09/2023**		
15	Write a shell program to implement a if statement.	**25/09/2023**		
16	Write a shell program to find factorial of any number.	**25/09/2023**		
17	Write a Shell program to check the given year is leap year or not	**28/09/2023**		
18	Write a Shell program to Swapping of two number	**28/09/2023**		
19	Write a Shell program to print table of any number.	**02/10/2023**		
20	Write a program to implement write operation using shared memory.	**04/10/2023**		
21	Write a program to implement read operation using shared memory.	**04/10/2023**		
22	Write a program to implement the concept of “paging”.	**11/10/2023**		
23	Write a program to simulation of FCFS CPU scheduling algorithm.	**11/10/2023**		
24	Write a program to simulation of Round Robin CPU scheduling algorithm.	**11/10/2023**		

Practical-1

Aim -: Understanding the booting process of Linux.

Press the power button on your system, and after few moments you see the Linux login prompt. From the time you press the power button until the Linux login prompt appears, the following sequence occurs. The following are the 6 high level stages of a typical Linux boot process.

Step 1. BIOS

• BIOS stands for Basic Input/Output System

• Performs some system integrity checks

• Searches, loads, and executes the boot loader program.

• It looks for boot loader in floppy, CD-ROMs, or hard drive. You can press a key (typically F12 or F2, but it depends on your system) during the BIOS startup to change the boot sequence.

• Once the boot loader program is detected and loaded into the memory, BIOS gives the control to it.

• So, in simple terms BIOS loads and executes the MBR boot loader.

Step 2. MBR

• MBR stands for Master Boot Record.

• It is located in the 1st sector of the bootable disk. Typically /dev/hda, or /dev/sda

• MBR is less than 512 bytes in size. This has three components 1) primary boot loader

• info in 1st 446 bytes 2) partition table info in next 64 bytes 3) mbr validation check in

• last 2 bytes.

• It contains information about GRUB (or LILO in old systems).

• So, in simple terms MBR loads and executes the GRUB boot loader.

Step 3. GRUB

● GRUB stands for Grand Unified Bootloader.

● If you have multiple kernel images installed on your system, you can choose which one to be executed.

● GRUB displays a splash screen, waits for few seconds, if you don't enter anything, it loads the default kernel image as specified in the grub configuration file.

● GRUB has the knowledge of the filesystem (the older Linux loader LILO didn't understand filesystem).

● Grub configuration file is /boot/grub/grub.conf (/etc/grub.conf is a link to this). The

following is sample grub.conf of CentOS.

Step 4. Kernel

- Mounts the root file system as specified in the “root=” in grub.conf
- Kernel executes the /sbin/init program
- Since init was the 1st program to be executed by Linux Kernel, it has the process id (PID) of 1. Do a ‘ps -ef | grep init’ and check the pid.
- initrd stands for Initial RAM Disk.
- initrd is used by kernel as temporary root file system until kernel is booted and the real root file system is mounted. It also contains necessary drivers compiled inside, which helps it to access the hard drive partitions, and other hardware.

Step 5. Init

●Looks at the /etc/inittab file to decide the Linux run level.

●Following are the available run levels

0 – halt

1 – Single user mode

2 – Multiuser, without NFS

3 – Full multiuser mode

4 – unused

5 – X11

6 – reboot

● Init identifies the default initlevel from /etc/inittab and uses that to load all appropriate program.

● Execute ‘grep initdefault /etc/inittab’ on your system to identify the default run level

● If you want to get into trouble, you can set the default run level to 0 or 6. Since you know what 0 and 6 means, probably you might not do that.

● Typically you would set the default run level to either 3 or 5.

Step 6. Runlevel programs

● When the Linux system is booting up, you might see various services getting started. For example, it might say “starting sendmail …. OK”. Those are the runlevel programs, executed from the run level direct

● on your default init level setting, the system will execute the programs from one of the following directories.

- Run level 0 – /etc/rc.d/rc0.d/
- Run level 1 – /etc/rc.d/rc1.d/
- Run level 2 – /etc/rc.d/rc2.d/

- Run level 3 – /etc/rc.d/rc3.d/
- Run level 4 – /etc/rc.d/rc4.d/
- Run level 5 – /etc/rc.d/rc5.d/
- Run level 6 – /etc/rc.d/rc6.d/

● Please note that there are also symbolic links available for these directory under /etc directly. So, /etc/rc0.d is linked to /etc/rc.d/rc0.d.

● Under the /etc/rc.d/rc*.d/ directories, you would see programs that start with S and K.

● Programs starts with S are used during startup. S for startup.

●Programs starts with K are used during shutdown. K for kill.

●There are numbers right next to S and K in the program names. Those are the sequence number in which the programs should be started or killed.

● For example, S12syslog is to start the syslog deamon, which has the sequence number of 12. S80sendmail is to start the sendmail daemon, which has the sequence number of 80. So, syslog program will be started before sendmail.

Login Process

1. Users enter their username and password

2. The operating system confirms your name and password.

3. A "shell" is created for you based on your entry in the "/etc/passwd" file

4. You are "placed" in your "home"directory.

5. Start-up information is read from the file named "/etc/profile". This file is known as the system login file. When every user logs in, they read the information in this file.

6. Additional information is read from the file named ".profile" that is located in your "home" directory. This file is known as your personal login file.

Practical-2

Aim -: Understand the behaviour of OS(Linux)and get the CPU type and model.

As everybody has questions while working with Linux OS, what are the features of Linux so we have enlisted the best characteristics of the Linux operating system.

1. Portable Environment

Linux software operates flawlessly on a variety of hardware platforms. Without the worry of incompatibility, individuals can use Linux operating system on any device. It runs the same way on both high-end and low-end hardware.

2. Free and Open-Source

Its source code is available for anybody to use and alter. Many developers collaborate in organizations to improve and strengthen Linux, and lots of developers constantly work on updating the Linux system.

3. Shell/ Command-line Interface

The Linux system includes essential programs that users can utilize in order to issue commands to the operating system for executing the design flawlessly. You may also direct it to carry out various forms of Linux commands for effectively carrying out the applications.

4. End-to-end encryption

Authentication can help you keep your data protected. Before you may access some critical files, the Linux Operating System requires you to enter a password. Furthermore, the Linux environment allows users to encrypt their data.

5. Graphical User Interface (GUI)

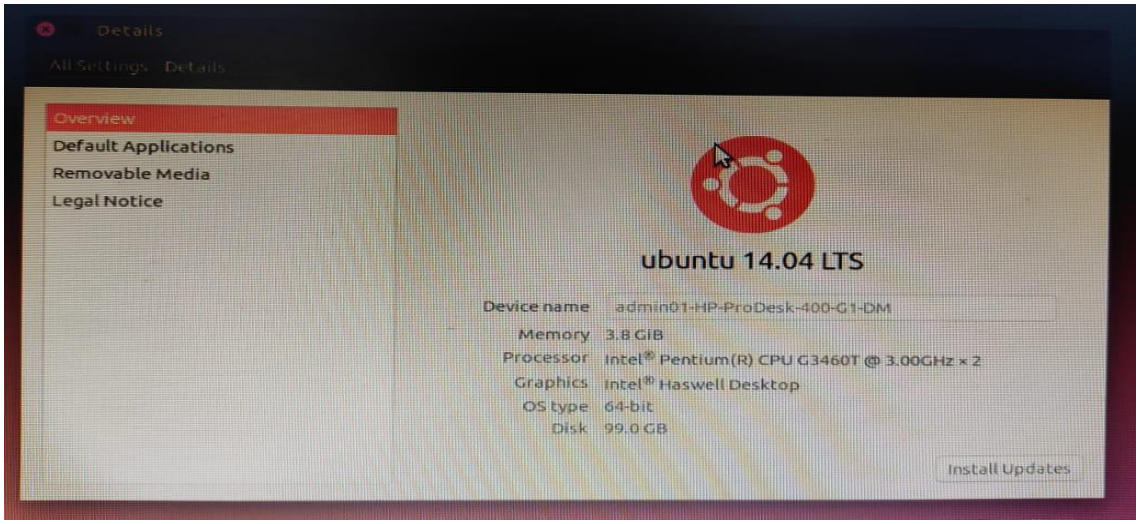
It comes with Graphical User Interface (GUI) abilities in the same way you can with Windows. Similarly, users can install the programs, and the computer graphics will begin to work in the same way that Windows does.CPU type and ModelSteps to know CPU type and Model in linux are –

Step1.Open Linux Operating system.

Step2. Open Setting (Present on top in right side).

Step3. About this Computer.

Step4. Overview


Details
All Settings Details
Overview
Default Applications
Removable Media
Legal Notice
ubuntu 14.04 LTS
Device name admin01-HP-ProDesk-400-G1-DM
Memory 3.8 GiB
Processor Intel® Pentium(R) CPU G3460T @ 3.00GHz × 2
Graphics Intel® Haswell Desktop
OS type 64-bit
Disk 99.0 GB
Install Updates

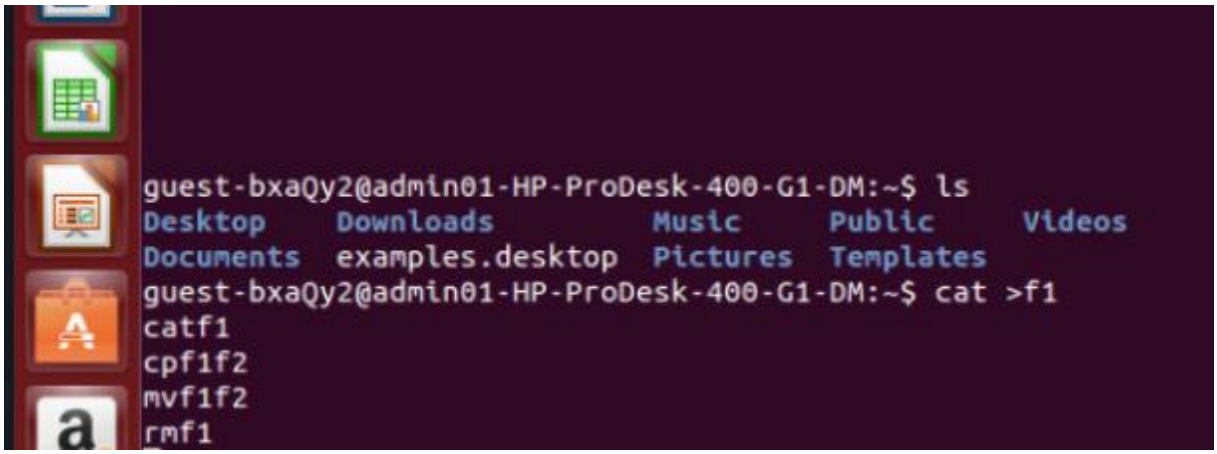
Practical-3

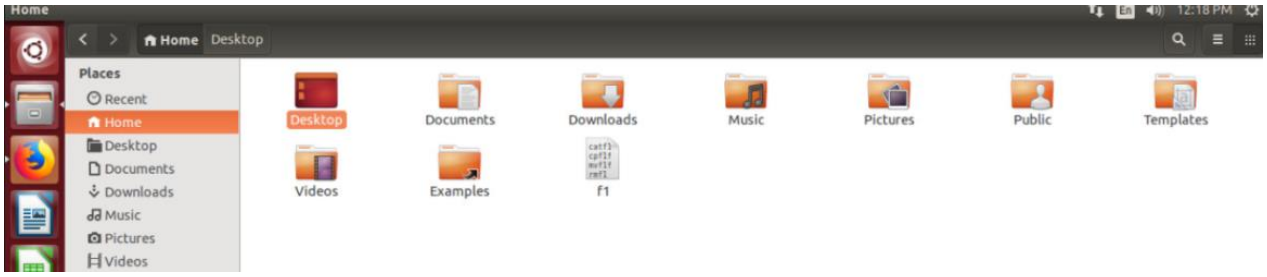
Aim:- Understanding the Linux file system commands.

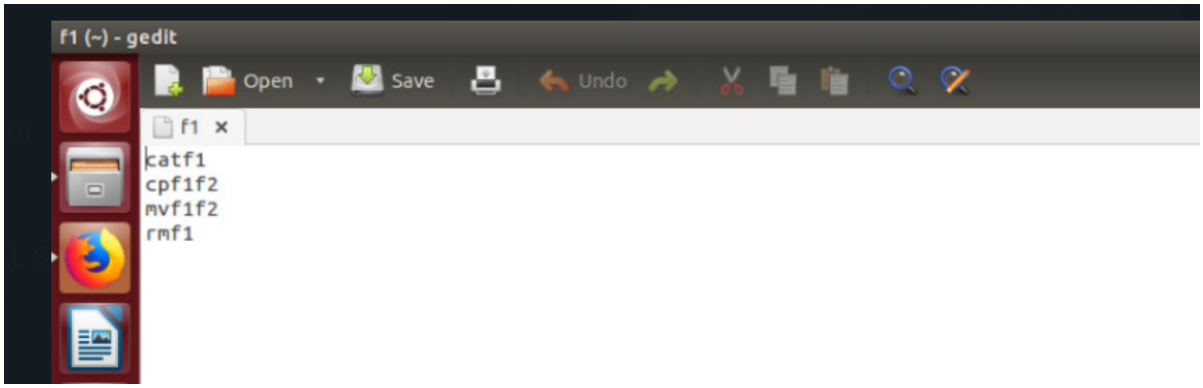
Syntax -:

1. ls - list files in the present working directory.
2. cat > f1 - to create a file (Press ^d to finish typing).
3. cat f1 - display the content of the file f1.
4. cp f1 f2- copy file f1 into f2.
5. mv f1 f2 - rename file f1 as f2.
6. rm f1 - remove the file f1 .

Output-






f1 (~) - gedit
Open
Save
Undo
f1
catf1
cpf1f2
mvf1f2
rmf1

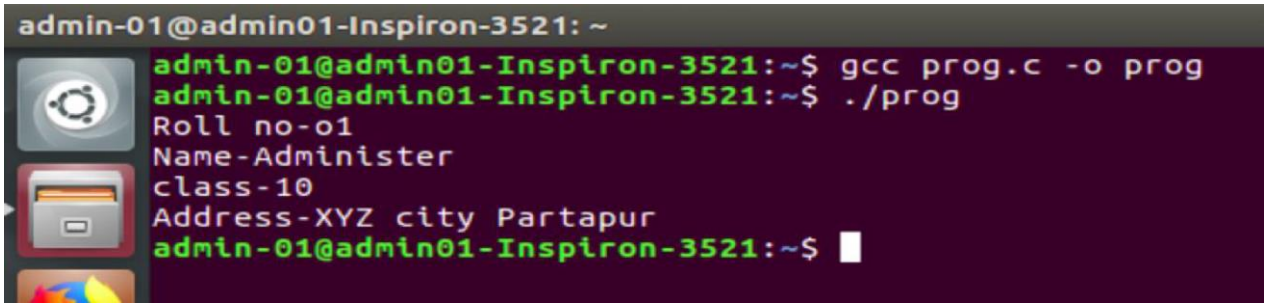
Practical-4

Aim: Write a program to read a file and display its content.

Program Code:

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<unistd.h>
#include<fcntl.h>
#include<stdlib.h>
#include<errno.h>
void main()
{
int fd,retval;
char buffer[400];
if((fd=open("data.dat", O_RDONLY))<0)
{
perror("problem in opening the file");
exit(1);
}
while((retval=read(fd,buffer,1))>0)
printf("%c",*buffer);
if(retval<0)
{
perror("\n Read failure");
exit(1);
} close(fd);
}
```

OUTPUT-


admin-01@admin01-Inspiron-3521: ~
admin-01@admin01-Inspiron-3521:~$ gcc prog.c -o prog
admin-01@admin01-Inspiron-3521:~$ ./prog
Roll no-o1
Name-Administer
class-10
Address-XYZ city Partapur
admin-01@admin01-Inspiron-3521:~$

Practical-5

Aim: Understanding the various phases of compilation and System admin commands.

Compilation in general is split into roughly 5 stages: Preprocessing, Parsing, Translation, Assembling, and Linking.

All 5 stages are implemented by one program in UNIX, namely cc, or in our case, gcc. The general order of things goes gcc ->gcc -E ->gcc -S -> as -> ld.

1.gcc

gcc is the C compiler of choice for most UNIX. The program gcc itself is actually just a front end that executes various other programs corresponding to each stage in the compilation process. To get it to print out the commands it executes at each step, use gcc v.

2. gcc -E (Preprocessor Stage)

gcc -E runs only the preprocessor stage. This places all include files into your .c file, and also translates all macros into inline C code.

3. gcc -S (Parsing+Translation Stages)

gcc -S will take .c files as input and output .s assembly files in AT&T syntax.gcc can be called with various optimization options that can do interesting things to the outputted assembly code. There are between 4 and 7 general optimization classes that can be specified with a -ON, where 0 <= N <= 6. 0 is no optimization (default), and 6 is maximum.

4. as (Assembly Stage)

as is the GNU assembler. It takes input as AT&T syntax asm files and generates a .o object file.

5. ld/collect2 (Linking Stage)

ld is the GNU linker. It will generate a valid executable file. If you link against shared libraries, you will want to actually use what gcc calls, which is collect2. Watch gcc -v for flags.

System Admin Commands:-

1. **ls**
   ls lists files in a directory.

2.cat

cat concatenates and prints files. You might issue cat to check the contents of your dependencies file.

3. grep

grep searches file patterns. If you are looking for a specific pattern in the output of another command, grep highlights the relevant lines.

```
root@kali:~/Documents# ls
abc  a.out  program17.c  program18.c
root@kali:~/Documents#
```

```
root@kali:~/Documents# cat abc
Srm IST
operating system
java

root@kali:~/Documents# grep operating abc
operating system
```

5. ps

The ps command, part of the procps-ng package which provides useful commands for investigating process IDs, shows the status of a running process.

```
$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  2 18:55 ?        00:00:02 /docker-java-home/jre/bi
root        59     0  0 18:55 pts/0    00:00:00 /bin/sh
root        75    59  0 18:57 pts/0    00:00:00 ps -ef
```

6. netstat

netstat shows the network status. This command shows network ports in use and their incoming connections.

```
[root@localhost ~]# netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      1/systemd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      823/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1053/master
tcp6       0      0 :::111                  :::*                    LISTEN      1/systemd
tcp6       0      0 :::80                   :::*                    LISTEN      18310/httpd
```

Practical-6

Aim-: Understanding the Linux commands.

1. clear - to clear the screen .

2. date - to display the current date and time.

3. cal 2003- to display the calendar for the year 2003 .

4. 4.cal 6 2003- to display the calendar for the June-2003.

5. passwd- to change password

6. ls -list files in the present working directory.

7. cat > f1 - to create a file (Press ^d to finish typing).

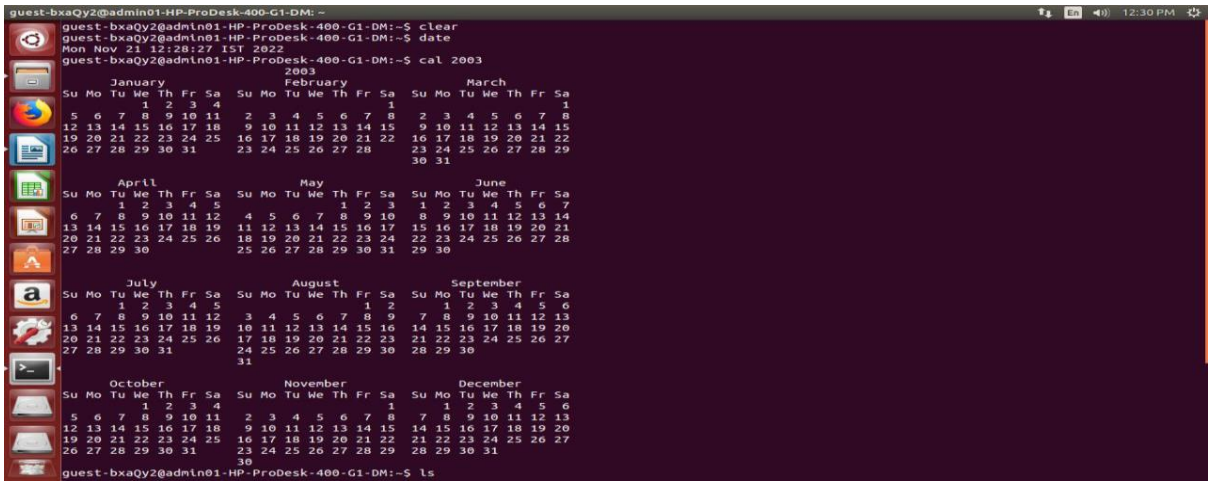
8. cat f1 - display the content of the file f1.

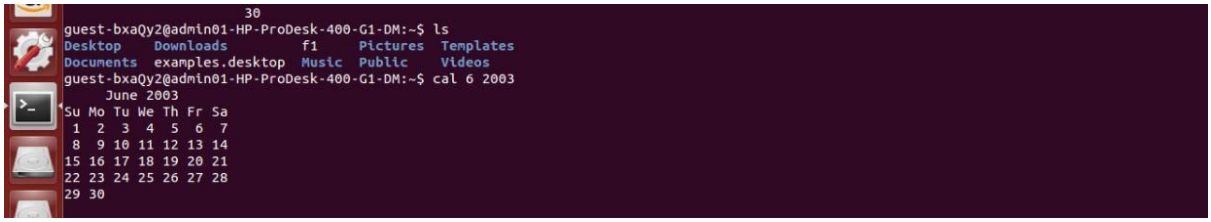
9. cp f1 f2- copy file f1 into f2.

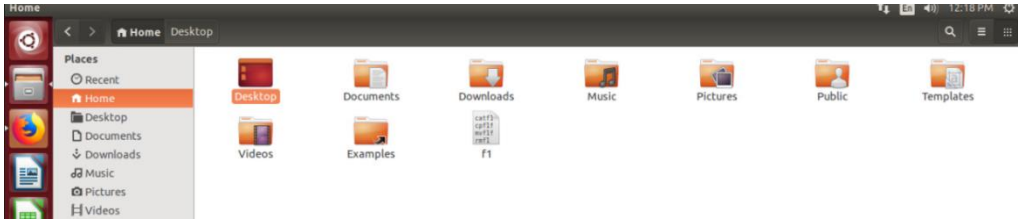
10.mv f1 f2 -rename file f1 as f2.

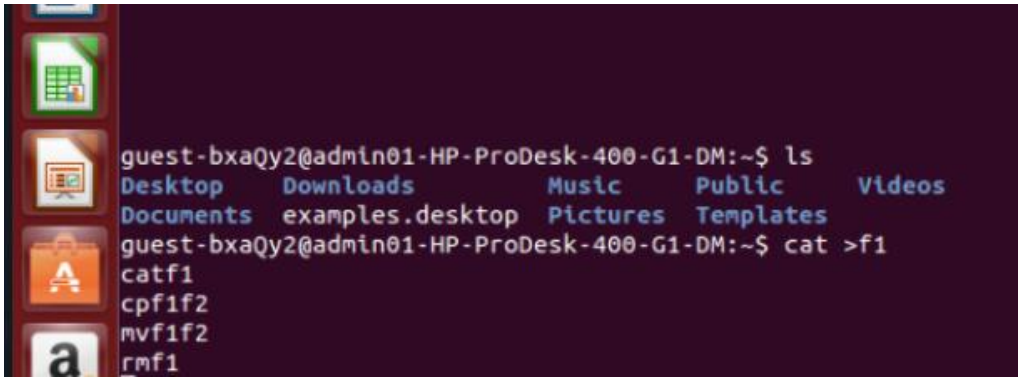
11.rm f1 - remove the file f1 .

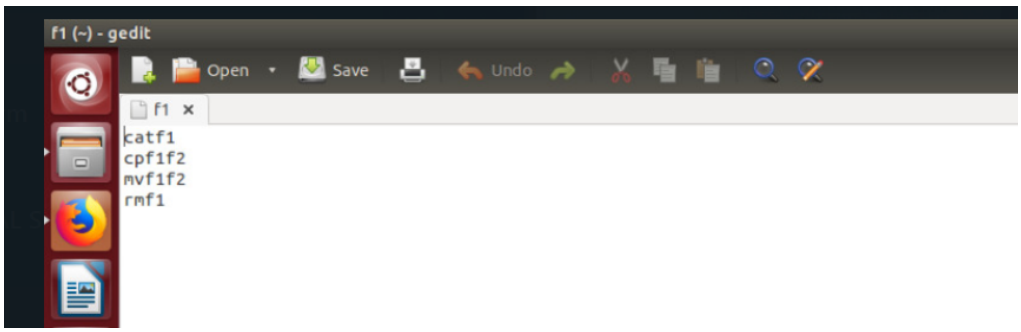
Output-








guest-bxaQy2@admin01-HP-ProDesk-400-G1-DM:~$ ls
Desktop Downloads Music Public Videos
Documents examples.desktop Pictures Templates
guest-bxaQy2@admin01-HP-ProDesk-400-G1-DM:~$ cat >f1
catf1
cpf1f2
mvf1f2
rmf1


f1 (~) - gedit
Open
Save
Undo
f1
catf1
cpf1f2
mvf1f2
rmf1

Practical-7

Aim -: Write a program to create child process using fork () system call.

Program -:

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<unistd.h>
int main(void) {
pid_tpid = fork();
if(pid == 0)
{
printf("child =>ppid: %d pid: %d\n" ,getppid(),getpid());
exit(EXIT_SUCCESS);
}
else if(pid> 0){
printf("parent =>pid: %d\n",getpid());
printf("waiting for child process to finish:\n");
wait(NULL);
printf("child process finished:\n");
}
else {
printf("unable to create child process:\n");
}
return EXIT_SUCCESS;
}
```

Output -:

```
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ touch fork.c
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ gcc fork.c
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ ./a.out fork.c
parent => pid: 2229
waiting for child process to finish:
child => ppid: 2229 pid: 2230
child process finished:
admin-01@admin01-HP-ProDesk-400-G1-DM:~$
```

Practical-8

Aim:- Write a program to implement the concept of thread.

Program -:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>
void *func(void *var)
{ sleep(2);
printf("we are using thread \n");
return NULL;
} int main()
{
pthread_tt_id;
printf("before using threading \n");
pthread_create(&t_id,NULL,func,NULL);
pthread_join(t_id,NULL);
printf("afterthread \n");
exit(0);
}
```

Output -:

```
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ gcc thread.c -lpthread
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ ./a.out
before using threading
we are using thread
afterthread
admin-01@admin01-HP-ProDesk-400-G1-DM:~$
```

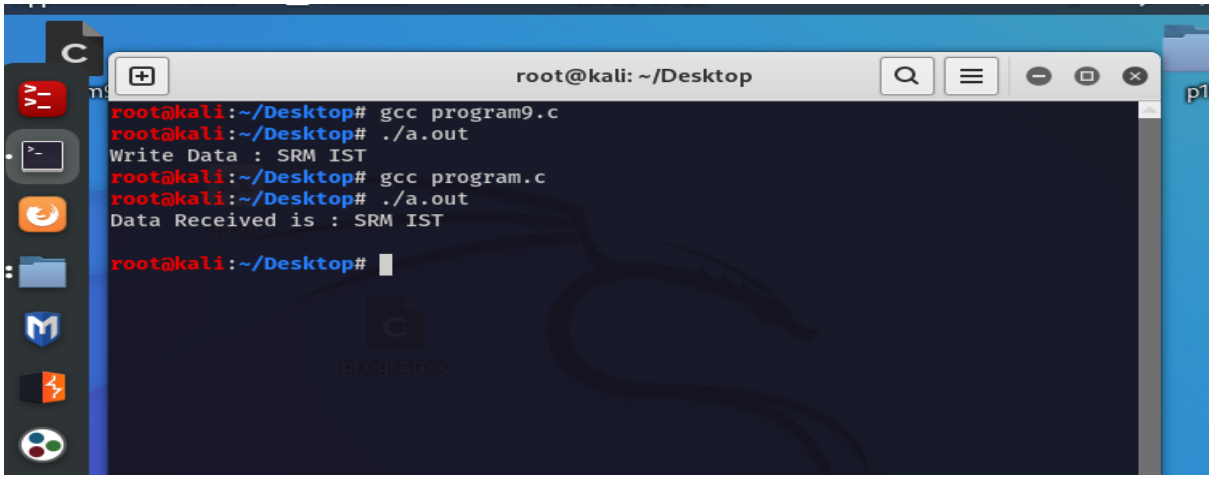
Practical-9

Aim-: Write a program to implement write operation of message passing in inter-process communication.

Program-:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MAX 10
struct mesg_buffer
{
long mesg_type;
char mesg_text[100];
}
message;
int main()
{
key_t key; int msgid;
key = ftok("profile", 65);
msgid = msgget(key, 0666 | IPC_CREAT); message.mesg_type = 1;
printf("Write Data : ");
fgets(message.mesg_text,MAX,stdin);
msgsnd(msgid, &message, sizeof(message), 0);
("Data send is : %s \n", message.mesg_text);
return 0;
}
```

Output:



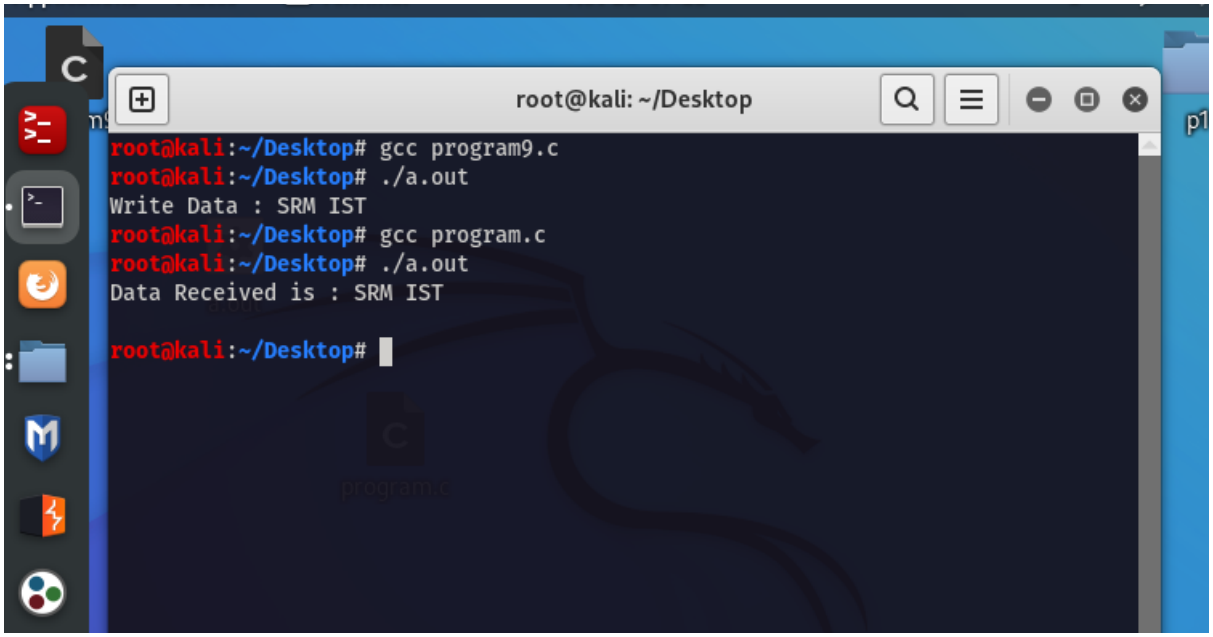
Practical-10

Aim-: Write a program to implement read operation of message passing in inter-process communication.

Program-:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct mesg_buffer
{
long mesg_type;
char mesg_text[100];
}
message;
int main()
{
key_t key; int msgid;
key = ftok("progfile", 65);
msgid = msgget(key, 0666 | IPC_CREAT);
msgrcv(msgid, &message, sizeof(message), 1, 0);
printf("Data Received is : %s \n", message.mesg_text);
msgctl(msgid, IPC_RMID, NULL);
return 0;
}
```

Output:



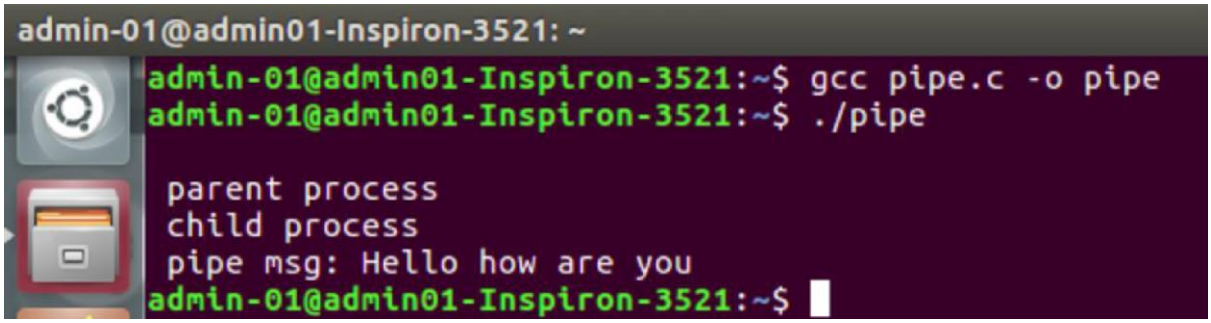
Practical-11

Aim-: Write a program to implement Pipe () function.

Program-:

```
#include <stdlib.h>

#include <stdio.h>

#include <unistd.h>

#include <string.h>

int main()

{

    int n;

    int filedes[2];

    char buffer[1025];

    char *message = "Hello, World!";

   pipe(filedes);

    write(filedes[1], message, strlen(message));

if ((n = read ( filedes[0], buffer, 1024 ) ) >= 0) {

        buffer[n] = 0;  //terminate the string

printf("read %d bytes from the pipe: "%s"\n", n, buffer);

    }

    else

perror("read");

exit(0);

}
```

Output-:


admin-01@admin01-Inspiron-3521: ~
admin-01@admin01-Inspiron-3521:~$ gcc pipe.c -o pipe
admin-01@admin01-Inspiron-3521:~$ ./pipe
parent process
child process
pipe msg: Hello how are you
admin-01@admin01-Inspiron-3521:~$

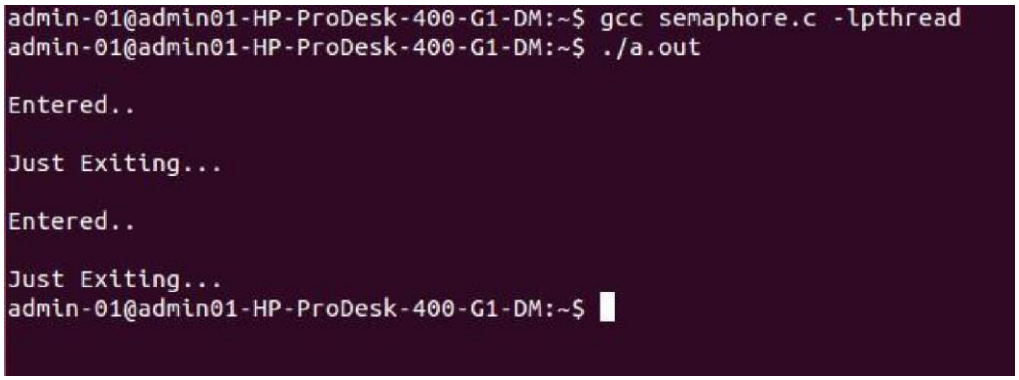
Practical-12

Aim-: To write a program to implement the concept of Semaphore.

Program-:

```
#include <stdio.h>
#include <pthread.h>
#include
<semaphore.h>
#include <unistd.h>
sem_t mutex;
void* thread(void* arg)
{ sem_wait(&mutex);
printf("\nEntered..\n")
sleep(4);
printf("\nJust
Exiting...\n");
sem_post(&mutex);
} int main()
{ sem_init(&mutex, 0,
1); pthread_t t1,t2;
pthread_create(&t1,NULL,thread,NULL
); sleep(2);
pthread_create(&t2,NULL,thread,NULL
); pthread_join(t1,NULL);
pthread_join(t2,NULL);
sem_destroy(&mutex);
return 0;
}
```

Output-:



```
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ gcc semaphore.c -lpthread
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ ./a.out

Entered..

Just Exiting...

Entered..

Just Exiting...
admin-01@admin01-HP-ProDesk-400-G1-DM:~$
```

Practical-13

Aim-: Write a program to implement the producer consumer problem.

Program-:

```
#include <stdio.h>
#include <stdlib.h>
int mutex = 1;
int full = 0;
int empty = 10, x = 0; void producer()
{
--mutex;
++full;
empty;
x++;
printf("\nProducer produces" "item %d",x);
++mutex;
}
void consumer()
{
--mutex;
--full;
++empty;
printf("\nConsumer consumes " "item %d",x); x--;
++mutex;
}
int main()
{
int n, i;
printf("\n1. Press 1 for Producer" "\n2. Press 2 for Consumer" "\n3. Press 3 for Exit");
#pragma omp critical for (i = 1; i> 0; i++) {
printf("\nEnter your choice:"); scanf("%d", &n);
switch (n)
{
case 1:
if ((mutex == 1) && (empty != 0)) { producer();
}
else
printf("Buffer is full!");
}
break; case 2:
if ((mutex == 1) && (full != 0)) { consumer();
}
else
{
printf("Buffer is empty!");
}
break;
case 3:
```

```
exit(0);
break;
}
}
}
```

Output-:

```
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ gcc producerconsumer.c
admin-01@admin01-HP-ProDesk-400-G1-DM:~$ ./a.out producerconsumer.c

1. Press 1 for Producer
2. Press 2 for Consumer
3. Press 3 for Exit
Enter your choice:1

Producer producesitem 1
Enter your choice:1

Producer producesitem 2
Enter your choice:1

Producer producesitem 3
Enter your choice:2

Consumer consumes item 3
Enter your choice:2

Consumer consumes item 2
Enter your choice:3
admin-01@admin01-HP-ProDesk-400-G1-DM:~$
```

Practical-14

Aim-: Write a shell program to implement While loop.

Program-:

```
#!/bin/sh
a=0
while [ $a -lt10 ]
do
  echo $a
  a=`expr $a + 1`
done
```

Output-:

```
0
1
2
3
4
5
6
7
8
9
```

Practical-15

Aim-: Write a shell program to implement if statement.

Program-:

Print Hello World

```
echo "Hello, World!"


# Command to Check WeatherYou are Above 18  or Not
echo "Enter your age:"
read age
if [ "$age" -ge18 ]; then
   echo "You are an adult."
else
   echo "You are a minor."
fi
#Command for checking number from 1 to 5.
echo "Counting from 1 to 5:"
for i in {1..5}; do
   echo $i
done
#Initializing two variables
a=10
b=20
#Check whether they are equal
if [ $a == $b ]
then
 echo "a is equal to b"
fi
```

```
#Check whether they are not equal

if [ $a != $b ]
then
 echo "a is not equal to b"
fi
```

Output-:

```
The variable 'b' is greater than the variable 'a'.
```

```
greet() {
    echo "Hello, $1!"
}

greet "Alice"
greet "Bob"
```

Practical-16

Aim-: Write a shell program to find factorial of any number.

Program-:

```
echo "Enter a number"
read num
fact=1
for((i=2;i<=num;i++))
{
  fact=$((fact * i))  #fact = fact * i
}
echo $fact
```

Output-:

```
Enter a number
3
6

Enter a number
4
24

Enter a number
5
120
```

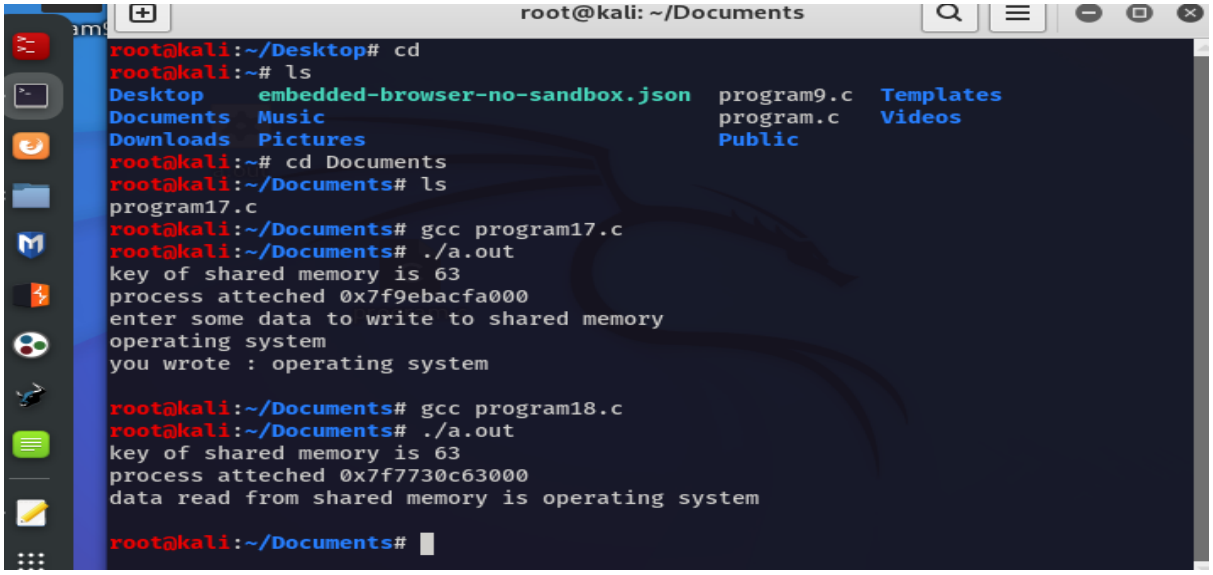
Practical-17

Aim-: Write a program to implement write operation using shared memory.

Program-:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
int i;
void *shared_memory;
char buff [100];
int shmid;
shmid=shmget((key_t)2345,1024,0666|IPC_CREAT);
printf("key of shared memory is %d\n",shmid);
shared_memory=shmat(shmid,NULL,0);
printf("process atteched %p\n",shared_memory);
printf("enter some data to write to shared memory \n");
read(0,buff,100);
strcpy(shared_memory,buff);
printf("you wrote : %s\n",(char *)shared_memory);
}
```

Output:



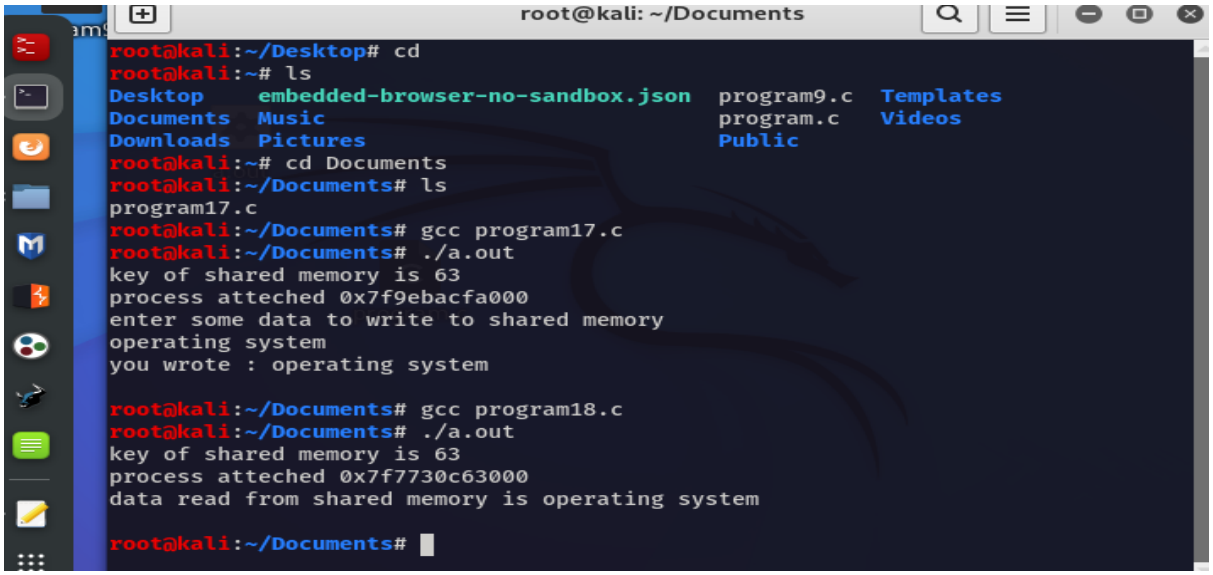
Practical-18

Aim-: Write a program to implement read operation using shared memory.

Program-:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
int i;
void *shared_memory;
char buff [100];
int shmid;
shmid=shmget((key_t)2345,1024,0666);
printf("key of shared memory is %d\n",shmid);
shared_memory=shmat(shmid,NULL,0);
printf("process atteched %p\n",shared_memory);
printf("data read from shared memory is %s\n",(char *)shared_memory);
}
```

Output:



Practical-19

Aim-: Write a program to implement the concept of "paging".

Program-:

```
#include<stdio.h>
#define MAX 50
int main()
{
int page[MAX],i,n,f,ps,off,pno;
int choice=0;
printf("\nEnter the no of pages in memory: ");
scanf("%d",&n);
printf("\nEnter page size: ");
scanf("%d",&ps);
printf("\nEnter no of frames: ");
scanf("%d",&f);
for(i=0;i<n;i++)
page[i]=-1;
printf("\nEnter the page table\n");
printf("(Enter frame no as -1 if that page is not present in any frame)\n\n");
printf("\npageno\tframeno\n-------\t-------");
for(i=0;i<n;i++)
{
printf("\n\n%d\t\t",i);
scanf("%d",&page[i]);
}
do
{
printf("\n\nEnter the logical address(i.e,page no & offset):");
scanf("%d%d",&pno,&off);
if(page[pno]==-1)
printf("\n\nThe required page is not available in any of frames");
else
printf("\n\nPhysical address(i.e,frame no & offset):%d,%d",page[pno],off);
printf("\nDo you want to continue(1/0)?:");
scanf("%d",&choice);
}while(choice==1);
return 1;
}
```

Output:

```
guest-q04wCm@admin17:~$ gcc paging.c
guest-q04wCm@admin17:~$ ./a.out

Enter the no of  peges in memory: 4

Enter page size: 10

Enter no of frames: 10

Enter the page table
(Enter frame no as -1 if that page is not present in any frame)


pageno  frameno
------- -------

0               -1


1               8


2               -1


3               6


Enter the logical address(i.e,page no & offset):2 200


The required page is not available in any of frames
Do you want to continue(1/0)?:1


Enter the logical address(i.e,page no & offset):1 500


Physical address(i.e,frame no & offset):8,500
```

Practical-20

Aim-: Write a program to simulation of FCFS CPU scheduling algorithm.

Program-:

```
#include<stdio.h>
void findWaitingTime(int processes[],int n,intbt[],int wt[])
{
wt[0] = 0;
for (int i = 1; i<n ;i++ )
wt[i] = bt[i-1] + wt[i-1] ;
}
void findTurnAroundTime( int processes[], int n,
int bt[], int wt[], int tat[])
{
for (int i = 0; i<n ;i++)
tat[i] = bt[i] + wt[i];
}
void findavgTime( int processes[], int n, int bt[])
{
int wt[n], tat[n], total_wt = 0, total_tat = 0;
findWaitingTime(processes, n, bt, wt);
findTurnAroundTime(processes, n, bt, wt, tat);
printf("Processes \t Burst time \t Waiting time \t Turn around time\n")
for (int i=0; i<n; i++)
{
total_wt = total_wt + wt[i];
total_tat = total_tat + tat[i];
printf(" %d ",(i+1));
printf(" %d ", bt[i] );
printf(" %d",wt[i] );
printf(" %d\n",tat[i] );
}
int s=(float)total_wt / (float)n;
int t=(float)total_tat / (float)n;
printf("Average waiting time = %d",s);
printf("\n");
printf("Average turn around time = %d ",t);
}
int main()
{
int processes[] = { 1, 2, 3};
int n = sizeof processes / sizeofprocesses[0];
int burst_time[] = {10, 5, 8};
findavgTime(processes, n, burst_time);
return 0;
}
```

Output

```
input
Processes        Burst time      Waiting time    Turn around time
 1       10      0       10
 2       5       10      15
 3       8       15      23
Average waiting time = 8
Average turn around time = 16

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical-21

Aim-: Write a program to simulation of Round Robin CPU scheduling algorithm.

Program-:

```
#include<stdio.h>
int main()
{
int count,j,n,time,remain,flag=0,time_quantum;
int wait_time=0,turnaround_time=0,at[10],bt[10],rt[10];
printf("Enter Total Process:\t ");
scanf("%d",&n);
remain=n;
for(count=0;count<n;count++)
{
printf("Enter Arrival Time and Burst Time for Process Process Number %d :",count+1);
scanf("%d",&at[count]);
scanf("%d",&bt[count]);
rt[count]=bt[count];
}
printf("Enter Time Quantum:\t");
scanf("%d",&time_quantum);
printf("\n\nProcess\t|TurnaroundTime|Waiting Time\n\n");
for(time=0,count=0;remain!=0;)
{
if(rt[count]<=time_quantum&& rt[count]>0)
{
time+=rt[count];
rt[count]=0;
flag=1;
} else if(rt[count]>0)
{
rt[count]-=time_quantum;
time+=time_quantum;
} if(rt[count]==0
&& flag==1)
{
remain--;
printf("P[%d]\t|\t%d\t|\t%d\n",count+1,time-at[count],time-at[count]-bt[count]);
wait_time+=time-at[count]-bt[count];
turnaround_time+=time-at[count];
flag=0;
} if(count==n-1)
count=0;
else if(at[count+1]<=time)
count++;
else
count=0;
```

```
}
printf("\nAverage Waiting Time= %f\n",wait_time*1.0/n);
printf("Avg Turnaround Time = %f",turnaround_time*1.0/n);
return 0;
}
```

Output

```
input
Enter Total Process:     4
Enter Arrival Time and Burst Time for Process Process Number 1 :3 5
Enter Arrival Time and Burst Time for Process Process Number 2 :2 6
Enter Arrival Time and Burst Time for Process Process Number 3 :1 5
Enter Arrival Time and Burst Time for Process Process Number 4 :4 6
Enter Time Quantum:     2


Process |Turnaround Time|Waiting Time

P[1]    |       14      |       9
P[2]    |       17      |       11
P[3]    |       19      |       14
P[4]    |       18      |       12

Average Waiting Time= 11.500000
Avg Turnaround Time = 17.000000

...Program finished with exit code 0
Press ENTER to exit console.
```

Practical 22

Aim: Write a Shell program to check the given year is leap year or not.

PROGRAM:

```
echo "Enter the year"
 read y
 b=`expr $y % 4`
 if [ $b -eq0 ]
 then
echo "$y is a leap year"
 else
echo "$y is not a leap year"
 fi
```

OUTPUT

Practical 23

Aim: Write a Shell program to Swapping of two number .

PROGRAM:

```
first=5
second=10

temp=$first
first=$second
second=$temp

echo "After swapping, numbers are:"
echo "first = $first, second = $second"
```

Output:

```
After swapping, numbers are:
first = 10, second = 5
```

Practical 24

Aim: Write a Shell program to print table of any number.

PROGRAM:

```
# Input from user
echo "Enter the number -"
read n

# initializing i with 1
i=1

# Looping i, i should be less than
# or equal to 10
while [ $i -le 10 ]
do
res=`expr $i \* $n`

# printing on console
echo "$n * $i = $res"

# incrementing i by one
((++i))

# end of while loop
done
```

Output:

```
Enter the number -
7
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
```